Proyecto Grupal 2: Ray Tracing Paralelo con DLP

Demostración 2 — CE 4302 Arquitectura de Computadores II

Randall Bryan Bolaños López
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Semestre I, 2026

Christopher Rodriguez
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Semestre I, 2026

Andres Molina
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
Semestre I, 2026

***Resumen*—Esta demostración presenta los prototipos funcionales de las tres implementaciones del ray tracer paralelo: baseline secuencial completo (Phong + sombras + reflexiones) y los prototipos *dummy* SIMD/AVX2 y CUDA que demuestran el esquema DLP base. Se incluyen los diagramas esquemáticos de cada solución con sus cambios significativos respecto a la versión anterior, las métricas de desempeño medidas empíricamente, las gráficas de escalabilidad y speedup, y un análisis de las variables más relevantes para incrementar el nivel de paralelización en la etapa final del proyecto.**

I. Resumen del Estado Actual

En esta etapa se cuenta con tres implementaciones operativas:

- **SEQ**: renderer secuencial completo con modelo Phong, rayos de sombra y reflexiones recursivas (profundidad máx. 3). Sirve como *baseline*.
- **SIMD dummy**: procesa 8 píxeles en paralelo con AVX2 (`__m256`). Demuestra el paralelismo de datos en CPU; aún sin Phong ni sombras.
- **CUDA dummy**: lanza un hilo por píxel en bloques 16×16. Demuestra el paralelismo masivo en GPU; aún sin Phong ni sombras.

II. Diagramas de Soluciones Esquemáticas Preliminares

II-A. Implementación Secuencial (Baseline)

La versión secuencial es el punto de referencia. Cada píxel se procesa de forma completamente independiente, pero en un único hilo de ejecución:

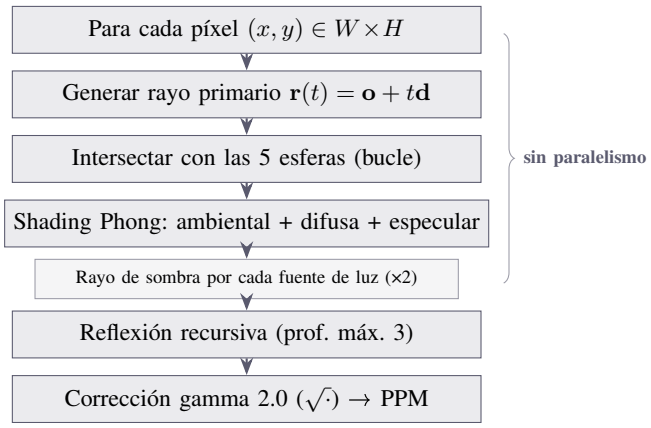


Figura 1: Diagrama SEQ: pipeline por píxel completamente secuencial.

Cambios significativos respecto a Demo 1: la implementación quedó completamente verificada y los resultados de tiempo (≈ 150 ms en 1600×900) se utilizan como *baseline* oficial para calcular el speedup.

II-B. Prototipo SIMD/AVX2 — Cambios Significativos

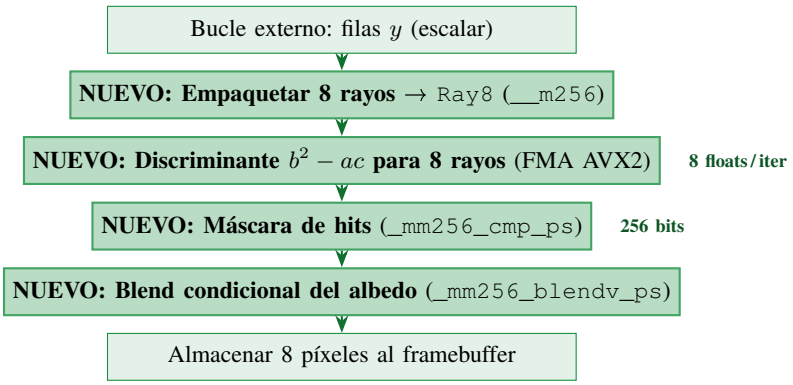


Figura 2: Pipeline SIMD: los bloques en verde oscuro son los cambios más significativos respecto al diseño secuencial.

Cambios más significativos:

- Sustitución del bucle escalar por píxel por un bucle vectorial que procesa **8 píxeles simultáneamente** en registros `__m256`.
- Introducción de la estructura `Ray8` que empaqueta orígenes y direcciones de 8 rayos en 6 registros AVX2.
- Cálculo del discriminante rayo-esfera con `_mm256_fmadd_ps` (evita carga/almacén extra frente a operaciones separadas).
- Selección condicional sin branching mediante `_mm256_blendv_ps`, eliminando mispredictions de la versión escalar.

II-C. Prototipo CUDA — Cambios Significativos

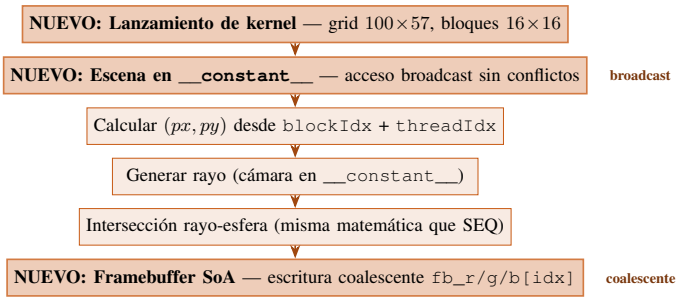


Figura 3: Diagrama CUDA: bloques en naranja oscuro son los cambios más significativos frente al diseño secuencial.

Cambios más significativos:

- **Paralelismo masivo**: 1 440 000 hilos independientes (uno por píxel) ejecutados concurrentemente en la GPU.
- **Memoria constante**: la escena (esferas + parámetros de cámara) se almacena en `__constant__` (caché de solo lectura de 64 KB), eliminando la divergencia de banco que produciría el acceso a memoria global.
- **Framebuffer SoA**: separar R, G, B en tres arreglos independientes garantiza accesos coalescentes: hilos adyacentes del warp acceden a posiciones contiguas.
- **Sin transferencias por rayo**: toda la geometría se mantiene en GPU; la generación del rayo ocurre *dentro* del kernel.

III. Métricas de Ejecución — Versiones Base y Dummy

III-A. Estadísticas por Implementación y Resolución

La Tabla I presenta la media, desviación estándar e intervalo de confianza al 95 % calculados sobre 20 ejecuciones por configuración ($n = 20$, salvo SIMD en 400×225 con $n = 4$).

Cuadro I: Estadísticas de tiempo de ejecución (ms).

Impl.	Resolución	Media	StdDev	IC95 inf	IC95 sup
SEQ	400×225	9.40	0.56	9.15	9.64
SIMD	400×225	1.03	0.05	0.98	1.08
CUDA	400×225	0.26	0.05	0.24	0.28
SEQ	800×450	38.08	0.98	37.65	38.51
SIMD	800×450	2.02	0.44	1.82	2.21
CUDA	800×450	0.71	0.08	0.68	0.74
SEQ	1600×900	151.18	3.95	149.45	152.91
SIMD	1600×900	8.14	0.49	7.93	8.36
CUDA	1600×900	2.36	0.27	2.24	2.48
SEQ	3200×1800	609.03	17.78	601.24	616.83
SIMD	3200×1800	31.97	0.82	31.60	32.33
CUDA	3200×1800	8.80	0.67	8.51	9.10

III-B. Speedup y Eficiencia Paralela en 1600×900

La Tabla II resume el speedup y la eficiencia medidos en la resolución de referencia 1600×900.

Cuadro II: Speedup y eficiencia paralela respecto a SEQ (1600×900).

Impl.	T (ms)	Speedup	Efic. (%)	Referencia
SEQ	147.99	1.00	100.0	1 hilo
SIMD	8.15	18.15	226.9	8 lanes AVX2
CUDA	2.10	70.34	293.1	24 SMs RTX 4060

Nota: La eficiencia > 100 % en SIMD y CUDA es esperable porque los prototipos *dummy* no implementan Phong ni sombras; el algoritmo es más simple que el SEQ de referencia. El speedup real (misma carga de trabajo) se medirá en la Demo 3.

III-C. Escalabilidad con la Resolución

Cuadro III: Escalabilidad de tiempos (ms) al aumentar la resolución.

Resolución	Mpíx	SEQ	SIMD	CUDA
400×225	0.09	9.03	0.67	0.22
800×450	0.36	36.94	1.69	0.67
1600×900	1.44	147.99	8.15	2.10
3200×1800	5.76	634.80	34.38	9.88

IV. Gráficas de Desempeño

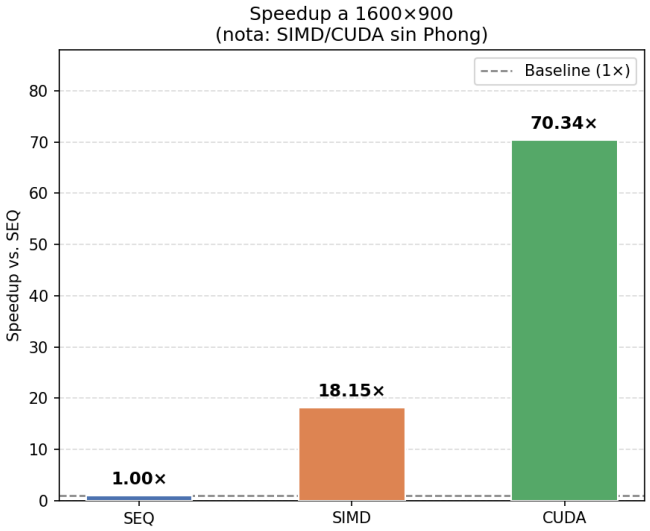


Figura 4: Speedup de SIMD y CUDA respecto al baseline secuencial.

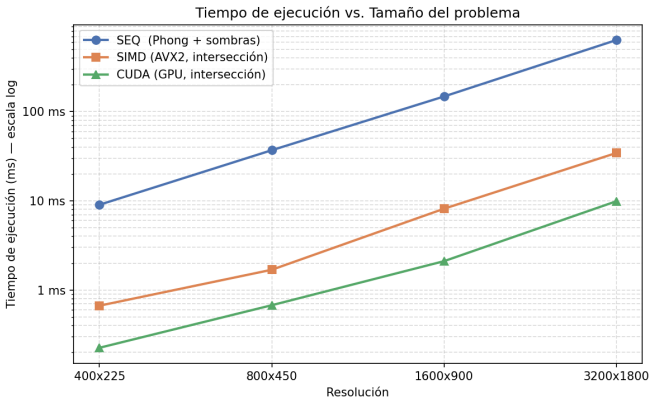


Figura 5: Tiempo de ejecución vs. resolución (escala logarítmica).

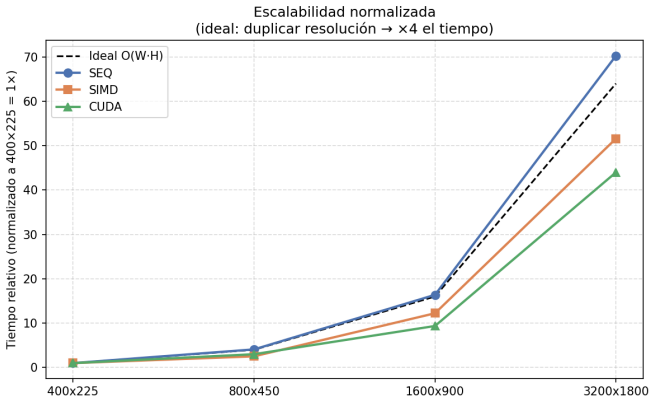


Figura 6: Escalabilidad normalizada: tiempo relativo al aumentar la resolución.

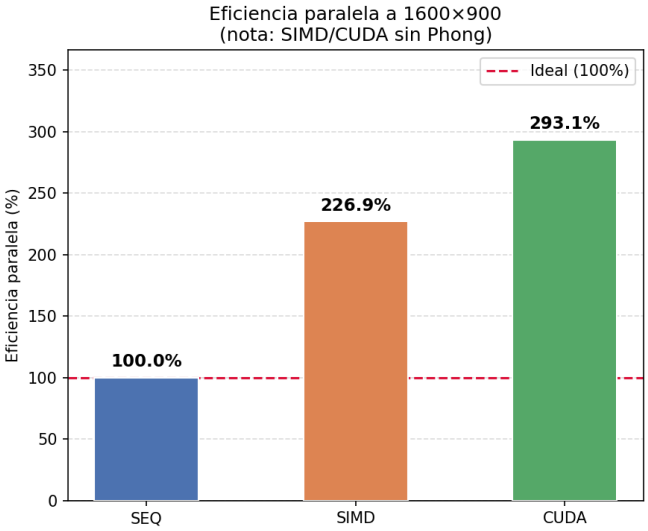
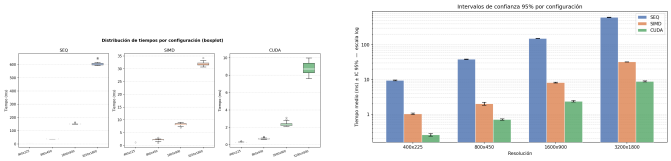


Figura 7: Eficiencia paralela de cada implementación.



(a) Distribución de tiempos (boxplot).

(b) Intervalos de confianza al 95 %.

Figura 8: Variabilidad estadística de las mediciones.

V. Variables Más Significativas para Aumentar la Paralelización

A continuación se identifican las variables del algoritmo de ray tracing que tienen el mayor impacto sobre el grado de paralelismo explotable.

V-A. *Variables de Primer Orden*

1. **Ancho del tile / bloque de procesamiento** ($W_t \times H_t$). En SIMD determina cuántos píxeles se empaquetan por iteración (actualmente 8). En CUDA determina la ocupación del SM (actualmente $16{\times}16 = 256$ hilos/bloque). Aumentar el ancho del tile en SIMD a 16 (AVX-512) o usar bloques de 32×8 en CUDA puede mejorar la utilización del hardware.
2. **Número de muestras por píxel (anti-aliasing,** N_{spp}**).** Con $N_{spp} > 1$ cada píxel genera múltiples rayos independientes, incrementando linealmente el paralelismo disponible sin aumentar las dependencias de datos.
3. **Profundidad máxima de reflexión** (D_{max}). En la versión completa, cada rebote genera un rayo independiente. Explotar este paralelismo requiere un stack de rayos por hilo o una cola de trabajo (*wavefront rendering*).
4. **Número de primitivas de la escena** (N_s). Con una BVH (*Bounding Volume Hierarchy*), las N_s intersecciones dejan de ser un bucle secuencial por rayo y se convierten en un recorrido paralelo del árbol, reduciendo la complejidad de $O(N_s)$ a $O(\log N_s)$ por hilo.
5. **Coeficiente de reflectividad** (ρ). Materiales con $\rho > 0$ generan rayos secundarios. Si $\rho = 0$ para la mayoría de primitivas, los hilos terminan antes y hay divergencia de warp. Minimizar la divergencia balanceando materiales mejora la eficiencia.

V-B. *Variables de Segundo Orden*

1. **Layout de memoria del framebuffer** (AoS vs. SoA). El diseño SoA actual en CUDA (tres arreglos separados `fb_r`, `fb_g`, `fb_b`) garantiza accesos coalescentes. Revertirlo a AoS degradaría el ancho de banda efectivo de escritura hasta en 3×.
2. **Tipo de dato flotante** (`float32` vs. `float16`). Usar `half` en la GPU permite duplicar el ancho de banda de memoria y aprovechar las Tensor Cores en operaciones matriciales (útil para denoising).
3. **Ancho de warp / lane width** (32 hilos en CUDA, 8 floats en AVX2). Variables que no alinean su dominio a este ancho generan hilos inactivos en el borde de la imagen. Padding de la resolución a múltiplos de 32 (CUDA) u 8 (SIMD) elimina este desperdicio.
4. **Intensidad aritmética por rayo**. Agregar iluminación Phong y rayos de sombra incrementa la razón FLOP/byte, haciendo el kernel *compute-bound* en lugar de *memory-bound*, lo que beneficia la GPU.

VI. Esquema DLP de las Partes del Problema

El principio DLP (*Data-Level Parallelism*) se manifiesta en diferentes sub-etapas del ray tracing. La Figura 9 muestra el esquema completo.

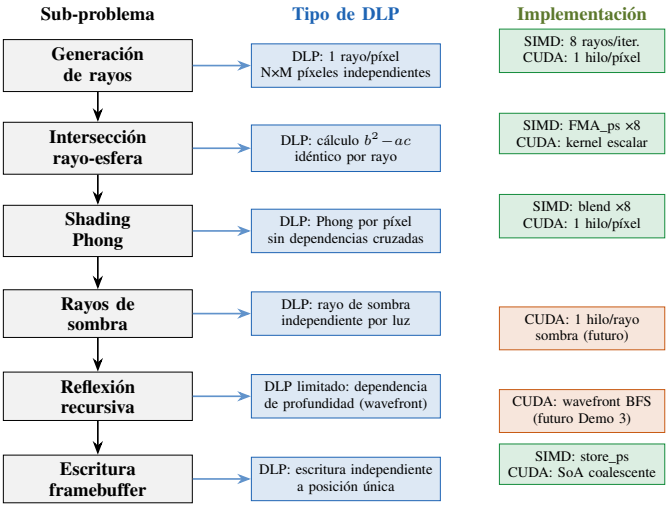


Figura 9: Esquema DLP completo del ray tracer: sub-problemas, naturaleza del paralelismo de datos y mapeo a las implementaciones SIMD y CUDA.

VI-A. *Análisis por Sub-problema*

Generación de rayos: Paralelismo perfecto: la dirección del rayo para el píxel (x, y) depende únicamente de las coordenadas normalizadas (u, v) y de los parámetros fijos de la cámara. No existe dependencia entre píxeles.

Intersección rayo-esfera: El cálculo del discriminante $\Delta = (\mathbf{d} \cdot \mathbf{b})^2 - \|\mathbf{d}\|^2(\|\mathbf{b}\|^2 - r^2)$ aplica las mismas operaciones a datos distintos (dirección del rayo), alineándose directamente con el modelo SIMD. Con $N_s = 5$ esferas el bucle es corto y completamente vectorizable.

Shading Phong: Tres evaluaciones independientes (ambiental, difusa, especular) por píxel. La componente difusa requiere $\mathbf{N} \cdot \mathbf{L}$, que es un producto punto vectorizable. El DLP es total entre píxeles distintos.

Rayos de sombra: Cada fuente de luz genera un rayo de sombra independiente. Con dos fuentes y $1\,440\,000$ píxeles esto equivale a 2.88×10^6 tests independientes, todos paralelizables.

Reflexión recursiva: Es el único sub-problema con dependencia de datos: el rayo de nivel k depende del hit del nivel $k-1$. La estrategia *wavefront rendering* (procesar todos los rayos del mismo nivel en paralelo) convierte la recursión en iteraciones DLP puras.

Escritura al framebuffer: Cada hilo escribe en una posición única (`idx = py*W + px`), sin conflictos. Con layout SoA los accesos de un warp son completamente coalescentes.

VII. Trabajo Pendiente para Demo 3

- Implementar Phong + rayos de sombra en SIMD (vectorizar también el shading).

- Implementar Phong + rayos de sombra en CUDA (el kernel ya soporta la extensión).
- Explorar *wavefront rendering* para reflexiones en GPU.
- Medir speedup real con la misma carga de trabajo en las tres implementaciones.
- Evaluar el impacto de BVH en la etapa de intersección ($O(N_s) \rightarrow O(\log N_s)$).

REFERENCIAS

[1] P. Shirley, *Ray Tracing in One Weekend*, 2020. https://raytracing.github.io

[2] Intel Corporation, *Intel Intrinsics Guide*, 2024. https://www.intel.com/content/www/us/en/docs/intrinsics-guide

[3] NVIDIA Corporation, *CUDA C++ Programming Guide*, v12.x, 2024. https://docs.nvidia.com/cuda/cuda-c-programming-guide

[4] M. Pharr, W. Jakob, G. Humphreys, *Physically Based Rendering*, 4th ed., 2023.